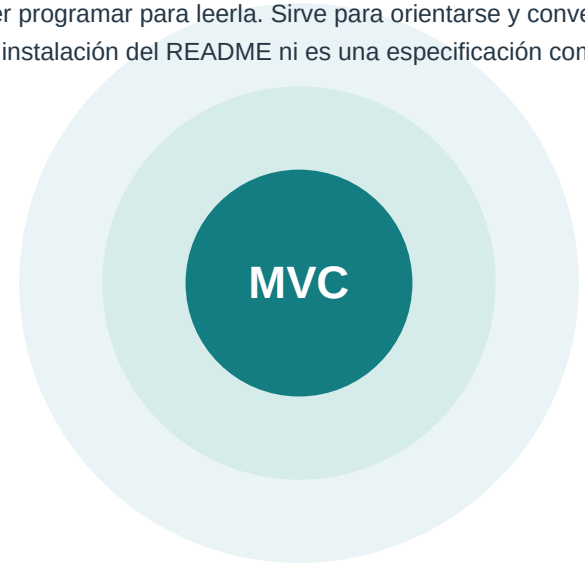
Cómo entender el código de Sistema FCT

Guía para personas sin experiencia técnica

Una visita guiada por las páginas, los datos y las decisiones que hacen funcionar el sistema.

Revisada el 14 de octubre de 2026

Esta guía explica, con palabras sencillas, cómo se organiza el proyecto y qué ocurre cuando una persona utiliza la aplicación. No hace falta saber programar para leerla. Sirve para orientarse y conversar con el equipo; no sustituye las instrucciones de instalación del README ni es una especificación completa del sistema.



1. La idea general: un equipo de trabajo

Una aplicación web se parece a una oficina que recibe solicitudes:

- **La persona usuaria** pide hacer algo desde una página: iniciar sesión, consultar una oferta o guardar sus datos.
- **La puerta de entrada** recibe la petición y la dirige al lugar adecuado. En este proyecto es `public/index.php`, que consulta el mapa de rutas de `routes/web.php`.
- **El controlador** entiende la petición y coordina el trabajo. Por ejemplo, comprueba los datos de un formulario y decide qué mostrar después.
- **El modelo** consulta o cambia la información guardada en la base de datos.
- **La vista** prepara la página que verá la persona.

Esta separación se llama **MVC**: Modelo, Vista y Controlador. No significa que cada cosa viva totalmente aislada; significa que hay responsabilidades reconocibles para que el proyecto sea más fácil de mantener.

Una forma rápida de orientarse:

Si quieres localizar...	Empieza por...
Las páginas y formularios	`app/Views/`
La lógica de una acción o formulario	`app/Controllers/`
Las consultas y cambios en los datos	`app/Models/`
Qué dirección web activa cada acción	`routes/web.php`
La conexión con MariaDB/MySQL	`config/database.php`
El diseño y las páginas públicas	`public/`
El componente interactivo de perfil de alumno	`frontend/src/islands/StudentProfile.vue`
El diálogo de confirmación para borrar cuentas	`frontend/src/islands/UserDeleteDialog.vue`

2. Un ejemplo real: cambiar el perfil de un alumno

Supongamos que un alumno corrige su teléfono:

1. El navegador solicita `index.php?action=perfil-alumno`.
2. La puerta de entrada busca `perfil-alumno` en `routes/web.php`. Allí se indica que solo puede entrar un alumno autenticado y que el controlador responsable es `AlumnoController`.
3. Antes de mostrar la página, `AuthMiddleware` comprueba que la sesión pertenece a una cuenta activa con el rol permitido.
4. `AlumnoController` obtiene los datos actuales usando modelos y pide mostrar `app/Views/alumnos/perfil.php`.
5. El formulario envía el nuevo teléfono mediante una petición POST. Incluye un código secreto de sesión llamado **token CSRF**, que ayuda a evitar que otra página envíe un formulario en nombre del usuario.

6 La aplicación vuelve a comprobar la sesión, el permiso, el token y los datos. La validación del servidor es la que decide si el dato se puede guardar; las comprobaciones de la página no la sustituyen.

7 El controlador pide al modelo que guarde el cambio. Si todo sale bien, la aplicación muestra la confirmación; si algo no es válido, vuelve a enseñar el formulario con un mensaje.

En este formulario concreto, Vue mejora la interacción en el navegador, pero el formulario HTML y PHP siguen existiendo. Por eso la lógica importante y el guardado no dependen exclusivamente de que JavaScript esté funcionando.

3. Las piezas del proyecto

Páginas y controladores

`app/Views/` contiene las plantillas de las pantallas: formularios, listados, mensajes y páginas de error. Las plantillas combinan HTML con PHP para insertar los datos que prepara el servidor.

`app/Controllers/` contiene clases que coordinan las acciones. Hay controladores para alumnos, empresas, ofertas, convenios, asignaciones, calificaciones, recuperación de cuenta y administración de usuarios, entre otros. Un controlador no debería decidir por sí solo quién puede acceder: los permisos de las rutas se declaran en `routes/web.php` y se aplican en la entrada de la aplicación.

`app/Core/View.php` localiza la plantilla solicitada y le entrega los datos para que la renderice. El diseño general y la navegación compartida se reúnen en `app/Views/layouts/main.php`.

Datos y modelos

`app/Models/` contiene las operaciones con los datos: encontrar un usuario, consultar una oferta, registrar una asignación o guardar una calificación. La conexión compartida se prepara en `config/database.php` con PDO, una herramienta de PHP para hablar con MariaDB/MySQL.

La base guarda información en tablas relacionadas. Por ejemplo, una cuenta está en `usuarios`; los datos propios de un alumno o una empresa se completan en sus tablas de perfil. Convenios, ofertas, disponibilidad, asignaciones y calificaciones representan partes distintas del proceso de prácticas.

El archivo `database/sistema_fct.sql` define la base inicial. Los archivos de `database/migrations/` describen cambios posteriores en orden. En Docker, los scripts de inicialización se aplican automáticamente cuando se crea una base en un volumen nuevo; no se vuelven a ejecutar automáticamente sobre un volumen existente.

Interfaz interactiva con Vue

La mayoría del sitio se entrega desde PHP. Vue se usa de manera progresiva, como una pequeña pieza interactiva dentro de una página, no como una aplicación separada que reemplace todas las páginas.

- `frontend/src/main.ts` busca en la página los elementos marcados para Vue y carga el componente que corresponda.
- `frontend/src/islands/StudentProfile.vue` implementa las mejoras interactivas del perfil del alumno.
- `frontend/src/islands/AvailabilitySchedule.vue` mejora la selección de días y horarios semanales del alumno; el formulario PHP sigue enviando y validando los datos.
- `frontend/src/islands/UserDeleteDialog.vue` muestra la confirmación al borrar una cuenta. La lista de usuarios y sus botones siguen generándose en PHP; el componente envía la petición protegida con CSRF y PHP vuelve a comprobar permisos y datos antes de borrar.

- `frontend/src/stores/session.ts` guarda en Pinia el estado de autenticación y el token CSRF. `AuthLoginEnhancement.vue` mejora el formulario existente con una petición JSON; `RoleDashboard.vue` muestra accesos adecuados al rol en la página de inicio.
- `index.php?action=session` comunica el usuario y rol activos con el formato JSON común. Las cookies de sesión siguen siendo gestionadas por PHP; la API no devuelve contraseñas ni hashes.
- `frontend/src/types/` define los tipos TypeScript de la respuesta JSON y de las entidades centrales; `frontend/src/api/client.ts` centraliza las peticiones HTTP del nuevo módulo.
- `app/Core/FrontendAssets.php` decide si la página carga Vue desde el servidor de desarrollo (HMR) o desde los archivos compilados.
- `npm run build` comprueba y genera los archivos listos para servir. `npm run typecheck` comprueba los tipos del código TypeScript.

Si Vue no está habilitado, el login conserva su envío HTML tradicional, la página de inicio mantiene su panel PHP y el perfil, horario y administración conservan sus fallbacks. En todos los casos, PHP sigue siendo responsable de la sesión, la autorización, CSRF y la validación del servidor.

Servicios que ejecutan el proyecto

`docker-compose.yml` describe los servicios de desarrollo:

- **app:** PHP y Apache, que reciben las peticiones web.
- **db:** MariaDB, que guarda las cuentas y los datos.
- **clamav:** servicio que analiza archivos subidos antes de aceptarlos.
- **frontend-dev / node:** herramientas opcionales de desarrollo y compilación del frontend.

Docker los inicia como piezas coordinadas. Los datos de la base y los archivos privados se conservan en volúmenes; por eso `docker compose down -v` puede borrarlos y debe usarse solo cuando se quiera eliminar expresamente esos datos.

4. Qué protege la información

Estas protecciones están en el código actual y son importantes para entender por qué algunas comprobaciones suceden más de una vez:

- **Permisos por rol:** las rutas declaran qué roles pueden acceder. Por ejemplo, el perfil propio se limita al rol Alumno; la gestión de usuarios está limitada a Administrador. El servidor comprueba el permiso, aunque se intente abrir una dirección directamente.
- **Cuenta activa:** las rutas protegidas vuelven a comprobar en la base de datos que la cuenta exista y siga activa.
- **Contraseñas:** se guardan como una huella criptográfica, no como texto legible. Al iniciar sesión, el sistema compara la contraseña introducida con esa huella.
- **Formularios protegidos:** las peticiones que cambian datos usan POST y se comprueba el token CSRF de la sesión.
- **Consultas a la base:** los modelos usan consultas preparadas para separar los datos introducidos por la persona de las instrucciones SQL.

- **Datos mostrados en páginas:** las vistas escapan el contenido antes de mostrarlo, reduciendo el riesgo de que un texto se interprete como código HTML.
- **Archivos privados:** el almacenamiento de documentos se mantiene separado de las páginas públicas. Las descargas pasan por controles de acceso y el sistema usa ClamAV para analizar cargas.
- **Errores:** los detalles técnicos se escriben en el registro del servidor; a la persona se le presenta un mensaje más general.

Estas medidas describen mecanismos concretos del proyecto; no constituyen una garantía de que una aplicación esté libre de errores ni sustituyen una auditoría y una configuración de producción adecuadas.

5. Cómo seguir el rastro de otra función

Cuando quieras entender una función, sigue este recorrido:

1. **Encuentra el nombre de la acción** en `routes/web.php`. La tabla indica dirección, método permitido, roles y controlador.
2. **Abre el controlador** indicado. Busca el método que aparece en la ruta; suele coordinar la lectura o el guardado.
3. **Busca el modelo** que el controlador utiliza. Allí puedes ver qué información consulta o modifica.
4. **Busca la vista** que el controlador renderiza. Ahí está el formulario o el resultado que verá la persona.
5. **Comprueba las protecciones relacionadas**: middleware, validación del servidor y, si se trata de archivos, servicio de almacenamiento y antivirus.
6. **Mira las pruebas** para saber qué comportamientos se han automatizado. `tests/file_security.php` comprueba controles de seguridad específicos; `tests/run.php` prueba varios flujos con una base de datos de pruebas desechable.

Ejemplos de rutas para empezar:

- `public/index.php` es la entrada web y organiza la ejecución de cada petición.
- `routes/web.php` es el mapa de las acciones y sus permisos.
- `app/Controllers/OfertaController.php` coordina acciones relacionadas con ofertas.
- `app/Models/Oferta.php` contiene operaciones de datos de ofertas.
- `app/Views/ofertas/` contiene pantallas de ofertas.

No todas las funciones siguen exactamente el mismo recorrido: algunos controladores gestionan varias pantallas y algunas operaciones pasan por más de un modelo.

6. Palabras que suelen confundir

Servidor: programa que recibe una petición y prepara una respuesta. Aquí PHP y Apache sirven las páginas.

Navegador: aplicación con la que una persona visita el sistema, como Chrome, Edge o Firefox.

Base de datos: conjunto organizado de información guardada en tablas, parecido a varias hojas relacionadas.

Ruta: nombre de una acción que el sistema reconoce, por ejemplo `action=ofertas`. No es necesariamente una carpeta del ordenador.

Sesión: memoria temporal del servidor que permite reconocer que una persona ya inició sesión.

Middleware: comprobación que se ejecuta antes de permitir una acción; aquí se usa, por ejemplo, para revisar sesión y rol.

Migración: cambio versionado en la estructura de la base de datos, como añadir una columna o tabla.

Build / compilación: preparación del código Vue y TypeScript en archivos que el navegador puede cargar.

HMR: actualización rápida de la interfaz durante el desarrollo. Sirve para programar y no es necesaria para usar la aplicación compilada.

Volumen Docker: espacio persistente donde Docker conserva datos entre reinicios. Borrar el contenedor no siempre borra los datos; borrar el volumen sí puede hacerlo.

7. Una precaución importante

El Compose del repositorio sirve para desarrollo y pruebas locales; el README contiene los pasos de puesta en marcha vigentes. No uses una base real para pruebas, no ejecutes la creación de cuentas demo en producción y no borres volúmenes para intentar corregir una migración. Para un despliegue real hacen falta, entre otras cosas, HTTPS, credenciales dedicadas, copias de seguridad comprobadas y una política de actualizaciones.

Si quieres regenerar este PDF

El archivo Markdown de esta misma carpeta es el texto fuente editable. Si cambia, se puede volver a generar el PDF con Python y ReportLab:

```
python -m pip install reportlab
python scripts/generar_guia_pdf.py
```

Documento de orientación basado en el código del repositorio y en su README. Si una instrucción operativa cambia, consulta el README actualizado.